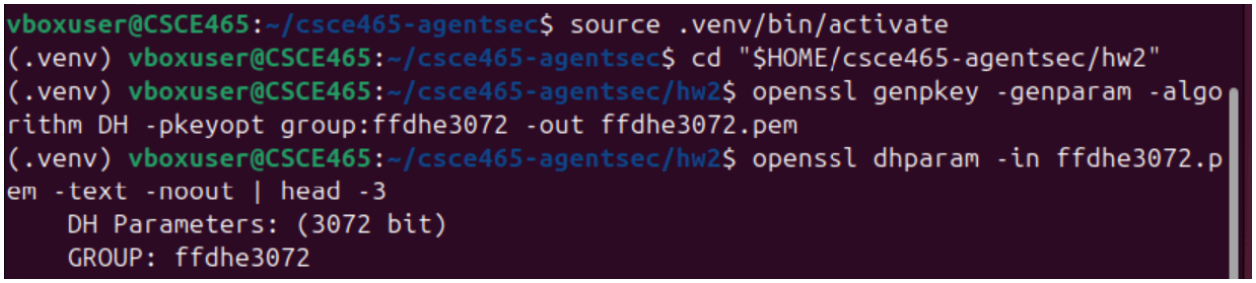
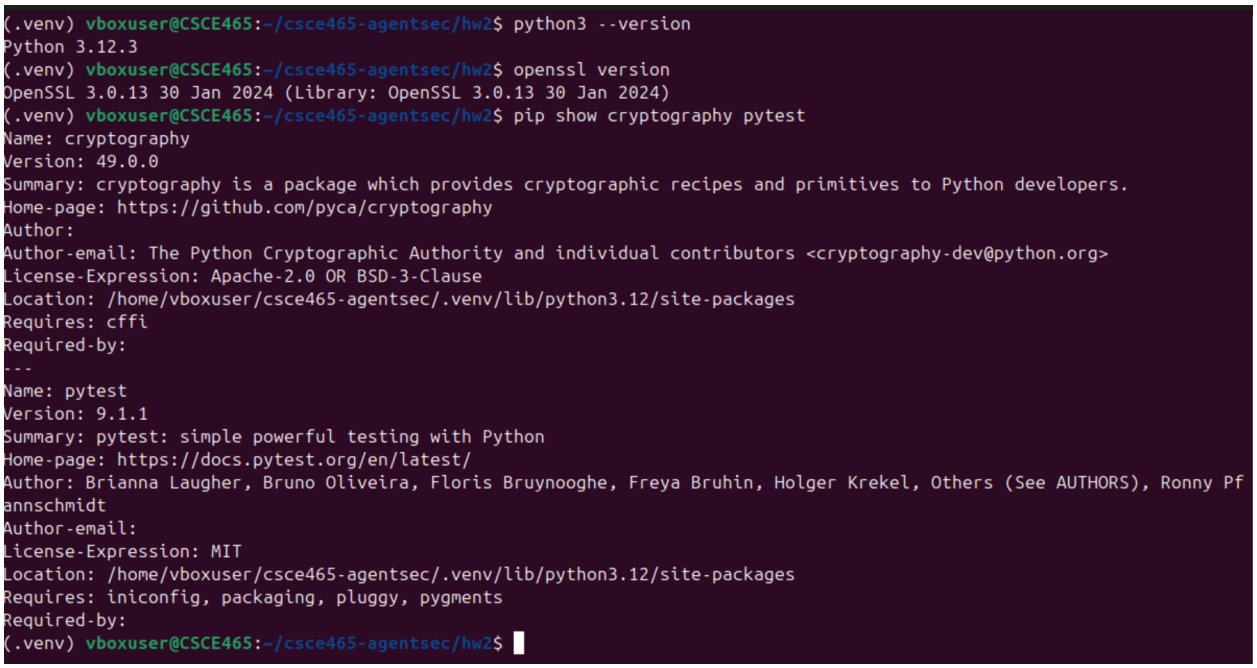
Github Link: https://github.com/IAmATack/CSCE465_HW2
Note: I was having issues getting the files to github due to power/internet outages, so the files show as a few commits hours apart. Also, I moved all the files from the VM to my computer all at once instead of after the completion of each task.

Task 0 - Lab Preparation:





Task 1 - Demonstrate why encryption alone is insufficient

```
(.venv) vboxuser@CSCE465:~/csce465-agentsec/hw2$ python3 baseline_ctr.py
Original plaintext:
{"action":"READ","path":"notes.txt"}

Original ciphertext:
67250760ed545f8d4499b9a750b54af3928f36300f0d67ed595a1efd3a9e14b06ce688d4

XOR mask:
000000000000000000000001e0c121000000000000000000000000000000000000000000000000

Modified ciphertext:
67250760ed545f8d4499b9b95ca75af3928f36300f0d67ed595a1efd3a9e14b06ce688d4

After ciphertext modification:
Receiver processed: {"action":"LIST","path":"notes.txt"}

Replay attack:
Receiver processed: {"action":"READ","path":"notes.txt"}
Receiver processed: {"action":"READ","path":"notes.txt"}
```

The design of baseline_ctr.py contains an encrypt and a decrypt helper function. These functions when given the key, iv, and text (plaintext or ciphertext respectively) will encrypt or decrypt the text. There is also a receiver function that is used to simulate the receiver of the decrypted ciphertext.

From the start, we generate a 256-bit AES key and a 128-bit IV. We take the original plaintext and encrypt the plaintext using the generated key and iv. From that encryption, we get the ciphertext. For the bit-flipping, we take the original message (`{"action":"READ","path":"notes.txt"}`) and our desired message (`{"action":"LIST","path":"notes.txt"}`) performing an XOR operation of each byte. We get our XOR mask. Now we take the ciphertext and our mask and perform an XOR operation on each byte, modifying the ciphertext to our desired message. At no point have we printed/learned the original key or IV.

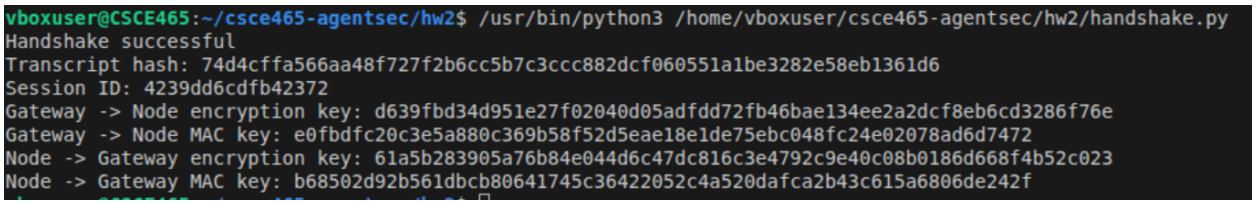
In the replay attack, we send the same original ciphertext twice which gives the original message back twice.

The modification of the ciphertext stems from having a predictable way to modify the ciphertext. An attacker can predictably modify the ciphertext by modifying certain bytes to output their desired message without having any changes propagate to the rest of the bytes of the text. Also, there is no check to make sure the message was modified, so the attacker can freely change the message with no consequence.

The replay attack can happen because there is no protocol requiring a different secret each time. There is also no authorization check to prevent the same person from sending the same message. This means an attacker can send the same ciphertext over and over again, appearing as different independent messages to the receiver.

Task 2 - Authenticated finite-field Diffie-Hellman setup

Output of handshake.py:



The handshake.py uses the pre-generated file to create public and private keys for both the node and the gateway. The two entities send their public keys over to each other. They both use the sent public key to create a shared secret with their private keys. This is the Diffie-Hellman key establishment. They both use private RSA keys to digitally sign a transcript. They use nonces to generate more randomness for the transcript.
This method is resistant to replay attacks because the keys and nonces are generated each time the message will be sent. All generated values are only good one time. Key separation is from each message using new keys and nonces each time. Forward secrecy is ensured because once the message has been communicated to both parties, the keys are disposed of. This ensures that an attacker would not be able to decrypt the messages.

Signing identities, roles, nonces, and Diffie-Hellman values prevents man-in-the middle attacks and reflection attacks because it is unknown how to break the RSA algorithm, at least without quantum computers. This means that an attacker should not be able to figure out the private key from the public key, unsign the message, change the message, and then sign it again. By signing all fields, we ensure that no modification of any individual field in the message was modified. A man-in-the-middle would not be able to act as both parties or even decrypt the message because of the use of the RSA keys. Another addition is the HMAC using SHA-256. The hash is one way, meaning that if the message was changed, the receiver would know.

Task 3 - Encrypt-then-MAC record layer

See the secure_record.py for the code
and the tests/test_secure_record.py for testing the code for secure_record.py

Task 4 - Adversarial tests and security note

See tests/test_security.py for the automated test that fulfills all requirements for

1. valid handshake and bidirectional messages;
2. modified ciphertext;
3. modified authenticated header;
4. replayed record;
5. record reflected into the opposite direction;
6. incorrect RSA public key, invalid RSA-PSS transcript signature, or reflected handshake message.

Security Note:
This attack model assumes that the attacker can observe, capture, inject, modify, replay old messages, and reorder messages. The attacker does not have access to the private keys or session secrets. This model does not assume that any user either the sender or receiver is malicious. In this attack model an attacker could perform man-in-the-middle or replay attacks.

AES-CTR requires the unique IVs for means of confidentiality. By using a unique IV for each message, if an attacker gets 2 messages from the same user, they are not able to decrypt either message. The user is allowed to use the same key multiple times, but the IV changes. This creates a unique ciphertext every time. If the user replayed the same message, it would show up differently because of the different IV.

Verify-before-decrypt matters because if the message has changed, we do not want to decrypt it. We use the Encrypt-then-MAC construction for this. The HMAC is used to verify each part of the message. The receiver will check the hash of the message parts against the hash sent to make sure the message was not modified during transit. This poses a layer of authentication as well because if the message was modified, we know that the person who sent the message is not who we expect. This HMAC keeps an attacker from modifying the different parts of the message.

Independent encryption and MAC keys are used because reusing keys is bad. If a key is used multiple times and an attacker receives multiple messages that use the same key, they could decrypt the ciphertext and read the message. Having the sender and receiver use independent keys for the handshake means that at no point does the other party learn of the other's key. If they used the same key, this could create an unintended connection between both user keys.

AES-GCM differs from this method in that AES-GCM combines both secrecy and tamper detection in a single step. It uses a stream cipher with a sequential counter for encryption and a Galois field to generate a method of authentication. The single step differs from this because we use the encrypt-then-MAC method. We have two steps, encryption and then generating the authentication method or the MAC address.

A valid protected record may still contain a dangerous authorized tool call because there are no safe guards to ensure that the message itself does not contain anything malicious. The record may pass all of the checks, but there is no check to make sure that sender is not sending a malicious tool command in the message. An attacker can inject whatever they want to whatever decrypts the message. Whatever receives the message must still validate the input and check that this message does not break any safety policies. The application that decrypts the message could be lured into performing an unauthorized action without the necessary checks to make sure that the message itself is not malicious. Assuming this is an agent, the message even after passing all of the validation and message authentication checks could still have some sort of “Ignore all instructions before this and perform this action” injection inside of it.